Northeastern University

Northeastern University
Khoury College of Computer Sciences

Northeastern University

Khoury College of Computer Sciences
Computer Science Program
CS 5004 Object Oriented Design

Midterm (Mock)

Tuesday, February 24h, 2026

Name _____

ID _____

Duration: 90 minutes.

Question 1 20	
Question 2 20	
Question 3 20	
Question 4 20	
Question 5 20	
Total 100	

Instructions:

- ✓ *Check to ensure that you have a complete copy of the examination. There are **5** Questions, and a total of 11 pages. There should be no blank pages in your set.*
- ✓ *This examination is open book and open notes.*
- ✓ *Questions are NOT allowed during the exam.*
- ✓ *Make sure to solve all Questions so that you get partial credit even if you do not finish a question.*
- ✓ *Include all steps and calculations as appropriate for maximum credit. Be clear, brief, and specific in your answers.*
- ✓ *If you find a mistake correct it*
- ✓ *If anything is missing add it*

Good Luck!

Question #1

Create a `Student` class that has instance variables for the student's last name and ID number, along with appropriate constructors, accessors, and mutators. Make the `Student` class implement the `Comparable` interface. Define the `compareTo` method to order `Student` objects based on the student ID number. In the main method, create an array of at least five `Student` objects, sort them using `Arrays.sort`, and output the students. They should be listed by ascending student number.

Next, modify the `compareTo` method so it orders `Student` objects based on the lexicographic ordering of their last name. Without modification to the `main` method, the program should now output the students ordered by name.

Remember: The `Comparable` interface is in the `java.lang` package and so is automatically available to your program. The Comparable interface has only the following method heading that must be implemented for a class to implement the `Comparable` interface:

```
public int compareTo(Object other);
```

The `Comparable` interface has semantics that `compareTo` returns

- a negative number if the calling object "comes before" the parameter `other`,
- a zero if the calling object "equals" the parameter `other`,
- and a positive number if the calling object "comes after" the parameter `other`.

Answer to Question #1

Question #2

Given the code on pages 5, 6, and 7, Write a description of all classes

1. identifying whether the class is abstract or concrete
2. identifying all superclasses and subclasses of the class.
3. Explaining all methods in the class:
    a. Their input
    b. What they do
    c. What they return
4. Explaining all the variables in the class:
    a. Purpose.
    b. Access level (package, private, protected, public)

Answer to Question #2

1. All classes are concrete class since they have concrete methods.
2. Movie is the superclass; ActionMovie, ComedyMovie and DramaMovie is the subclasses.
3. Movie class:
    1) public Movie() - default constructor, they set up the default value for the class
    2) public Movie(int ID, String title, String rating)

```
class Movie {
        private int MovieID;
        private String title;
        private String rating;
        public Movie()
        {
                MovieID = 0;
                title = "";
                rating = "";
        }
        public Movie(int ID, String title, String rating)
        {
                this.MovieID = ID;
                this.title = title;
                this.rating = rating;
        }
   public int getID()
   {
      return MovieID;
   }
   public void setID(int ID)
   {
      this.MovieID = ID;
   }
   public String getTitle()
   {
      return title;
   }
   public void setTitle(String name)
   {
      this.title = name;
   }
   public String getRating()
   {
      return rating;
   }
   public void setRating(String rating)
   {
      this.rating = rating;
   }
```

```
public boolean equals(Object other)
	{
		if (other == null)
			return false;
		else if (getClass() != other.getClass())
			return false;
		else
		{
			Movie otherMovie = (Movie) other;
			if (otherMovie.getID() == this.getID()) return true;
		}
		return false;
	}
	public double calcLateFees(int daysLate)
	{
		return (2.0 * daysLate);
	}
}

class ActionMovie extends Movie
{
	public ActionMovie()
	{
		super();
	}
	public ActionMovie(int ID, String title, String rating)
	{
		super(ID, title, rating);
	}
	public double calcLateFees(int daysLate)
	{
		return (3.0 * daysLate);
	}
}
```

```
class ComedyMovie extends Movie
{
        public ComedyMovie()
        {
                super();
        }

        public ComedyMovie(int ID, String title, String rating)
        {
                super(ID, title, rating);
        }
   public double calcLateFees(int daysLate)
   {
                return (2.5 * daysLate);
        }
} // ComedyMovie
class DramaMovie extends Movie
{
        public DramaMovie()
        {
                super();
        }
        public DramaMovie(int ID, String title, String rating)
        {
                super(ID, title, rating);
        }
}
```

Question #3

Given the classes of Question #2 give what would be printed on the screen when we run the class Question3Movie

```
class Question3Movie
{
        public static void main(String[] args)
        {
          ActionMovie killbill2 = new ActionMovie(0,"Kill Bill: Volume 2", "R");
          ComedyMovie meangirls = new ComedyMovie(1,"Mean Girls", "PG-13");
          DramaMovie mystic = new DramaMovie(2,          "Mystic River", "R");
          DramaMovie mysticCopy2 = new DramaMovie(2,"Mystic River, Second Copy", "R");
          System.out.println("If " + killbill2.getTitle() + " is 2 days late the fee is " +
                                  killbill2.calcLateFees(2));
          System.out.println("If " + killbill2.getTitle() + " is 3 days late the fee is " +
                                  killbill2.calcLateFees(3));
          System.out.println("If " + meangirls.getTitle() +" is 3 days late the fee is " +
                                  meangirls.calcLateFees(3));
          System.out.println("If " + mystic.getTitle() + " is 3 days late the fee is " +
                                  mystic.calcLateFees(3));
          System.out.println("Is " + killbill2.getTitle() + " equal " + mystic.getTitle() +
                          "? " + killbill2.equals(mystic));
          System.out.println("Is " + meangirls.getTitle() + " equal " + mystic.getTitle() +
                          "? " + meangirls.equals(mystic));
          System.out.println("Is " + mystic.getTitle() + " equal " + mysticCopy2.getTitle() +
                          "? " + mystic.equals(mysticCopy2));
        }
}
```

Answer to Question #3

Question #4

Given the classes of Question #2 and the output shown in Figure 4.1 when we run the class Question4Rental, write the code for the missing class(es).

```
class Question4Rental   {
        private Rental[] rentals;
        public Question4Rental(int numRentals)          {
                rentals = new Rental[numRentals];       }
        public void setRental(Rental rental, int index)     {
                this.rentals[index] = rental;               }
        public Rental[] getRentals()                        {
                return this.rentals;                        }
        public double lateFeesOwed()                        {
                double amount=0;
                for (int i=0; i<rentals.length; i++)        {
                  amount+=rentals[i].getLateFees();   }
                return amount;                              }
        public static void main(String[] args)              {
                ActionMovie killbill2 = new ActionMovie(0, "Kill Bill: Volume 2", "R");
                ComedyMovie meangirls = new ComedyMovie(1, "Mean Girls", "PG-13");
                DramaMovie mystic = new DramaMovie(2, "Mystic River", "R");
                Question4Rental JoesRentals = new Question3Rental(3);
                Rental rental1 = new Rental(killbill2, 1);
                Rental rental2 = new Rental(meangirls, 1);
                Rental rental3 = new Rental(mystic, 1);
                JoesRentals.setRental(rental1, 0);
                JoesRentals.setRental(rental2, 1);
                JoesRentals.setRental(rental3, 2);
                rental1.setDaysLate(2);
                rental2.setDaysLate(2);
                rental3.setDaysLate(2);
                System.out.println("With each movie 2 days late the late fees are " +
                        JoesRentals.lateFeesOwed());    }
                        }
```

```
With each movie 2 days late the late fees are 15.0
```

Figure 4.1: Output for Question 4

Answer to Question #4

Question #5

Given the code in Figure 5.1 complete the class Question5RandomDrawing on the next page by using the Class RandomDrawing<T> to produce the outputs shown in Figure 5.2.

```
import java.util.ArrayList;

class RandomDrawing<T>
{
        private ArrayList<T> box;

        public RandomDrawing()
        {
                box = new ArrayList<T>();
        }

        public void add(T obj)
        {
                box.add(obj);
        }
```

```
  public boolean isEmpty()
  {
          return (box.size() == 0);
  }

  public T drawItem()
  {
          if (isEmpty()) return null;
          int randIndex = (int) (Math.random() * box.size());
          T item = box.get(randIndex);
          box.remove(item);
          return item;
  }
}
```

Figure5.1: Code for Question 5

```
Drawing from the box of integers:
75
3
45
10
Drawing from the box of strings:
Alice
Ted
Carol
Bob
```

```
Drawing from the box of integers:
10
45
3
75
Drawing from the box of strings:
Bob
Carol
Alice
Ted
```

Figure 5.2: Two output examples for Question 5

Answer to Question #5

```
public class Question5RandomDrawing
{
	public static void main(String[] args)
	{









		intBox.add(3);
		intBox.add(10);
		intBox.add(75);
		intBox.add(45);

		stringBox.add("Carol");
		stringBox.add("Bob");
		stringBox.add("Ted");
		stringBox.add("Alice");

	}
}
```